Projet de Fin de TP – À Rendre

À partir du projet réalisé durant le TP, vous devez choisir et implémenter les améliorations ci-dessous. Chaque partie est notée indépendamment. Le projet est noté sur 20 points.

> Rendu : lien GitHub obligatoire + README.md expliquant comment lancer le projet + fichier .env.example sans clés réelles.

Partie	Intitulé	Points	Difficulté
A1	Base de données MySql/Postgres	2 pts	★ Facile
A2	Cours boursiers réels (yfinance)	2 pts	★ Facile
A3	Recherche web (TavilySearch)	2 pts	★★ Moyen
B1	Calcul de portefeuille boursier	2 pts	★★ Moyen
B2	PythonREPLTool	4 pts	★★★ Difficile
C1	Interface Streamlit	2 pts	★ Facile
C2	Mémoire conversationnelle	2 pts	★★ Moyen
D1	Api Rest Python + tool + database	4 pts	★★★ Difficile
	TOTAL	20 pts	

A1 – Base de Données SQLite (2 pts)

Migrez les dictionnaires CLIENTS et PRODUITS de database.py vers une vraie base de données MySql/Postgres en utilisant le module Python sqlite3. La structure de l'agent ne doit pas changer.

- Créer un fichier **database.db** avec les tables clients et produits
- Écrire un script init_db.py qui crée les tables et insère les données initiales
- Remplacer les fonctions rechercher_client et rechercher_produit par des requêtes SQL
- Vérifier que l'agent fonctionne exactement comme avant

A2 – Cours Boursiers Réels avec yfinance (2 pts)

Remplacez les variations aléatoires de finance.py par de vraies données de marché récupérées via la bibliothèque yfinance. Gardez la même interface de sortie.

- Installer yfinance (déjà dans les dépendances disponibles)
- Remplacer obtenir_cours_action pour utiliser yfinance.Ticker
- Afficher le cours réel, la variation du jour et le volume
- Gérer les erreurs si le symbole est invalide ou l'API indisponible

A3 – Recherche Web avec TavilySearch (2 pts)

Ajoutez un 7ème outil basé sur TavilySearchResults pour permettre à l'agent de répondre à des questions ouvertes : actualités financières, informations sur une entreprise, cours récents non couverts par les autres outils.

- Créer un compte gratuit sur tavily.com et obtenir une clé API
- Ajouter TAVILY_API_KEY dans le fichier .env
- Importer TavilySearchResults depuis langchain_community.tools
- Intégrer l'outil dans la liste tools de agent.py avec une description adaptée
- Tester avec des questions comme : actualités Apple aujourd'hui, résultats trimestriels LVMH

D1 - Agent d'analyse de portefeuille financier (4 pts)

Un agent Claude reçoit des questions en langage naturel sur un portefeuille d'investissement ("Quels sont mes actifs les plus risqués ?").
Il interroge une base de données PostgreSQL contenant les positions, prix et historiques de marché.
Le tout est exposé via une API REST (FastAPI/Express) avec des endpoints comme POST /api/agent/query.
Le clients appelle l'endpoint et reçoit une réponse structurée en JSON.

B1 – Calcul de Portefeuille Boursier (2 pts)

Créez un nouvel outil dans tools/portefeuille.py qui calcule la valeur totale d'un portefeuille d'actions en récupérant les cours réels via yfinance.

- La fonction prend en entrée une liste d'actions au format SYMBOLE:QUANTITE séparés par |
- Elle récupère le cours actuel de chaque action via yfinance
- Elle retourne la valeur de chaque ligne, la valeur totale et la variation globale du jour

B2 – PythonREPLTool (4 pts)

Intégrez PythonREPLTool de langchain_experimental comme outil supplémentaire. Cet outil permet à l'agent d'écrire et d'exécuter du code Python directement pour effectuer des calculs complexes non couverts par les outils existants.

- Importer PythonREPLTool depuis langchain_experimental.tools
- L'ajouter à la liste tools avec une description claire de son usage
- Tester avec des calculs complexes : tri d'un portefeuille, statistiques
- Documenter les risques de sécurité dans un commentaire dans le code

```
from langchain_experimental.tools import PythonREPLTool

python_repl = PythonREPLTool()
python_repl.description = (
    'Exécute du code Python pour des calculs complexes ou traitements '
    'de données non couverts par les autres outils. '
    'Entrée : code Python valide sous forme de chaîne.'
)

# ATTENTION SECURITE : cet outil exécute du code arbitraire.
# Ne jamais utiliser en production sans sandbox.
```

C1 – Interface Streamlit (2 pts)

Créez un fichier app.py qui expose l'agent via une interface web Streamlit. L'agent doit être utilisable depuis un navigateur sans passer par le terminal.

- Un champ de saisie en bas de page pour poser une question
- L'historique des échanges affiché dans la page (question + réponse)
- Un panneau latéral (sidebar) qui affiche les outils disponibles
- Un bouton pour réinitialiser la conversation

C2 – Mémoire Conversationnelle (2 pts)

Remplacez l'agent sans mémoire par un agent utilisant ConversationBufferMemory. Démontrez le fonctionnement avec un scénario en 3 questions liées où le contexte est conservé.

- Modifier **creer_agent()** pour intégrer **ConversationBufferMemory**
- Utiliser **create_openai_tools_agent** à la place de create_react_agent
- Créer un scénario de démonstration en 3 questions enchaînées
- La 3ème question doit nécessiter les informations des 2 premières pour être résolue

```
# Scénario de démonstration attendu :

# Q1 : 'Donne-moi les infos du client Sophie Bernard'
# → L'agent récupère le profil VIP, solde 28 900€

# Q2 : 'Quel produit lui recommandes-tu ?'
# → L'agent se souvient que c'est une cliente VIP

# Q3 : 'Calcule le prix TTC et dis-moi si elle peut se le permettre'
# → L'agent se souvient du produit ET du solde
```